PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

# Как модель учится на примерах

## Содержание


- 1. Вступление
- 2. Главная идея: чем ИИ-модель отличается от обычной программы?
- 3. Ключевые понятия
- 4. Как происходит обучение: пошаговый разбор
- 5. Учимся видеть: примеры, признаки, метки
- 6. От карточек – к таблицам данных
- 7. Как это выглядит в Python
- 8. Памятка
- Материалы к занятию


## 1. Вступление

**Добро пожаловать в Модуль 3!**
Ты уже умеешь работать с вероятностями, считать среднее и медиану, строить гистограммы. Ты знаешь, что такое ошибка модели и как её измерить. Теперь – главный вопрос:
**Как вообще ИИ-модель «знает», что делать? Кто её научил? И как именно происходит это обучение?**
Ответ на эти вопросы – тема всего Модуля 3. А начнём мы с самого фундаментального: как модель учится на примерах.
Именно примеры связывают исходные данные с нужным результатом: модель получает характеристики объектов и правильные ответы, а затем ищет между ними устойчивые связи.
**Что ты узнаешь на этом уроке:**

- Что такое модель ИИ – и чем она отличается от программы с готовыми правилами.
- Что такое обучающий пример, признак и правильный ответ.
- Как из примеров рождается решение.

- Почему «больше примеров» – не всегда «лучше модель».

К концу урока ты сможешь посмотреть на простую таблицу данных и определить, где находятся объекты, признаки и метки, а также объяснить, зачем данные разделяют на обучающие и тестовые.

## 2. Главная идея: чем ИИ-модель отличается от обычной программы?

Представьте, что вы хотите написать программу, которая определяет, хорошая ли погода для прогулки.
Трудность состоит в том, что подходящие условия не всегда удаётся точно перечислить заранее. Решение может зависеть сразу от температуры, дождя, ветра, влажности и их сочетаний.

**Обычная программа (правила вручную)**

```
if temperature > 15
and rain == False
and wind < 20:
    answer = 'хорошая погода'
else:
    answer = 'плохая погода'
```

- Правила написал человек.
- Нужно знать ВСЕ условия заранее.
- Если условия меняются – код переписывают.
- Одинаковые входные значения всегда проверяются по одним и тем же явно заданным условиям.

**ИИ-модель (обучение на примерах)**

- Людям показывают 1000 описаний погоды.
- И говорят: «вот хорошая», «вот плохая».
- Модель САМА находит закономерности: «При >15° и без дождя – обычно хорошая».
- Правила не пишут – модель их выводит.
- Можно учить на новых примерах.
- Работает с тысячами признаков.
- Результат обучения хранится во внутренних параметрах модели, а не обязательно в виде понятного человеку списка условий.

> **Ключевая аналогия**
>
> **Обычная программа** – это повар, который строго следует рецепту. Если в рецепте ошибка – блюдо испорчено.

> **ИИ-модель** – это повар, который попробовал тысячи блюд и научился чувствовать вкус. Он готовит «на интуиции» — но эта «интуиция» основана на огромном опыте (примерах).
> **Важно:** «интуиция» модели – это математика, а не магия. Просто математика очень сложная.
> При этом ИИ-модель тоже является программой. Отличие не в отсутствии кода, а в том, что часть правил поведения формируется во время обучения на данных. Если примеры были ошибочными или однообразными, математическая «интуиция» модели тоже окажется ненадёжной.

# 3. Ключевые понятия

Чтобы описывать обучение точно, важно различать сам объект, его характеристики и ответ, который требуется получить. Эти элементы выполняют разные роли, хотя в датасете часто находятся рядом.

| Понятие | Простое объяснение | Пример из жизни | Пример в ИИ |
|---|---|---|---|
| **Модель ИИ** | Программа, которая умеет делать прогнозы или принимать решения, опираясь на данные | Опытный врач, который ставит диагноз по симптомам | Система, которая определяет: спам это письмо или нет |
| **Обучающий пример** | Одна строка данных с известным правильным ответом | Одно письмо, которое человек пометил как «спам» | Строка в таблице: текст письма + метка «спам/не спам» |
| **Признак (feature)** | Одна характеристика объекта, которую модель может «видеть» | Симптом: температура, кашель, насморк | Длина письма, есть ли слово «выиграй», адрес отправителя |
| **Метка / правильный ответ** | То, что мы хотим, чтобы модель предсказала | Диагноз: «грипп» или «не грипп» | 1 = спам, 0 = не спам |
| **Обучение (training)** | Процесс, в ходе которого модель «изучает» закономерности из примеров | Студент-медик смотрит тысячи историй болезни | Алгоритм перебирает тысячи писем и подстраивает внутренние параметры |
| **Обучающая выборка** | Набор обучающих примеров, на которых тренируется модель | Учебник с разобранными задачами | 10 000 помеченных писем в базе данных |
| **Тестовая выборка** | Примеры, которые модель НЕ видела во время обучения – для проверки | Экзаменационные задачи (не из учебника) | Новые 1 000 писем для проверки точности |

> **Не путайте объект, признак и метку**
>
> В задаче фильтрации почты **объект** — целое письмо. Его **признаки** — длина, отдельные слова и сведения об отправителе. **Метка** — итоговый класс «спам» или «не спам».
> Один объект обычно описывается несколькими признаками, но в конкретной задаче имеет одну заданную метку.

# 4. Как происходит обучение: пошаговый разбор

Разберём на конкретном примере: обучение модели для классификации фотографий «кот / не кот».
**Классификация** означает, что модель должна выбрать один из заранее определённых классов. В данном случае классов два: «кот» и «не кот».

## Шаг 1. Собрать обучающие примеры

**Пример: датасет фотографий**
Обучающая выборка:

- Фото 1: [пушистый, маленький, острые уши, усы] → КОТ
- Фото 2: [гладкий, большой, круглые уши, нет усов] → НЕ КОТ
- Фото 3: [пушистый, маленький, острые уши, нет усов] → КОТ
- ... ещё 10 000 фотографий ...

Каждый пример: **ПРИЗНАКИ** (что мы видим) + **МЕТКА** (правильный ответ). Метку ставит человек – это называется «разметка данных».
Чтобы выборка отражала задачу, в ней нужны разные изображения обоих классов: при разном освещении, фоне, размере и положении объекта. Важно также проверять разметку: если фотографию кота случайно отметить как «не кот», модель получит противоречивый обучающий сигнал.

## Шаг 2. Модель ищет закономерности

Модель «смотрит» на тысячи примеров и ищет: какие признаки чаще встречаются у котов? Она учитывает не только отдельные признаки, но и их сочетания. Один признак может быть неоднозначным: например, острые уши встречаются не только у котов.
**Что «замечает» модель**

- «Острые уши» → 94% случаев это кот

- «Усы длиннее 3 см» → 87% случаев это кот
- «Пушистый + маленький + острые уши» → 98% случаев это кот

Эти закономерности модель не получает готовыми — она САМА находит их, перебирая миллионы комбинаций. Именно поэтому нужно много примеров.
Приведённые проценты иллюстрируют принцип: в реальной задаче значения зависят от конкретного датасета и способа обучения. Закономерность полезна только тогда, когда она повторяется и помогает правильно классифицировать не только знакомые, но и новые фотографии.

## Шаг 3. Модель делает прогноз на новом фото

После обучения метка нового объекта заранее неизвестна. Модель получает только его признаки и должна вычислить прогноз.
**Новый объект (без метки)**
Приходит новое фото: [пушистый, маленький, острые уши, усы]
Модель сравнивает с выученными закономерностями:

- «Пушистый» – часто кот: +0,4 в пользу КОТ
- «Маленький» – часто кот: +0,3 в пользу КОТ
- «Острые уши» – почти всегда кот: +0,8 в пользу КОТ
- «Усы» – очень характерно для кота: +0,7 в пользу КОТ

**Итого: вероятность КОТ = 96% → Прогноз: КОТ**
Знаки «+» здесь показывают идею вклада признаков, а не универсальную формулу, одинаковую для всех моделей. Вероятность 96% означает высокую уверенность модели, но не гарантирует правильность ответа. Проверить прогноз можно только по настоящей метке фотографии.

## Шаг 4. Проверка на тестовой выборке

**Зачем нужна тестовая выборка**
Представьте: вы учились по учебнику и запомнили все задачи наизусть. На экзамене точно такие же задачи – вы всё решаете. Но если задачи изменились – вы не справляетесь.
Это называется **«переобучение» (overfitting)**: модель «зазубрила» обучающие примеры, но не поняла общую закономерность.
**Тестовая выборка – это «экзамен на новых задачах».** Если модель хорошо работает и на ней – обучение прошло успешно.
Во время обучения тестовые примеры нельзя показывать модели, иначе проверка перестаёт быть независимой. Качество сравнивают по известным меткам тестовой выборки: прогноз

модели сопоставляют с правильным ответом для каждого объекта.
Если результат на обучающей выборке высокий, а на тестовой заметно хуже, это один из признаков переобучения.

# 5. Учимся видеть: примеры, признаки, метки

Посмотрим на три разных задачи машинного обучения. В каждой – своя «таблица обучающих примеров». Попробуй найти закономерность сам, до того как модель её выучит.
При разборе каждой таблицы сначала определите строки с известными метками, затем перечислите признаки нового объекта и найдите наиболее похожие обучающие примеры.
Такой способ не заменяет настоящий алгоритм, но помогает понять логику обучения.

## Задача А. Спам или не спам?

| Обучающий пример | Слово «выиграй»? | Длина письма | Знакомый адрес? | Много заглавных? | МЕТКА |
|---|---|---|---|---|---|
| Письмо 1 | да | короткое | нет | да | Спам |
| Письмо 2 | нет | длинное | да | нет | Не спам |
| Письмо 3 | да | короткое | нет | да | Спам |
| Письмо 4 | нет | длинное | да | нет | Не спам |
| Письмо 5 | нет | среднее | нет | нет | Не спам |
| **Письмо 6 (новое)** | да | короткое | нет | да | ??? |

**Разбор**

«Слово «выиграй»», «короткое», «незнакомый адрес», «много заглавных» → в обучающих примерах это сочетание всегда = Спам. Модель предскажет: **Спам**.
Письмо 6 полностью совпадает по указанным признакам с Письмами 1 и 3, поэтому вывод в этой учебной таблице однозначен.
Какой признак самый важный? Вероятно, «Слово «выиграй»» – во всех спам-примерах оно есть. Однако по пяти обучающим строкам нельзя доказать, что этот признак всегда будет главным. В большем датасете слово может встретиться и в обычном письме, поэтому модель должна учитывать сочетание признаков.

## Задача Б. Одобрить кредит или нет?

| Клиент | Возраст | Работает? | Долги есть? | Доход (тыс.) | МЕТКА |
|---|---|---|---|---|---|
| Клиент 1 | 35 | да | нет | 60 | Одобрить |
| Клиент 2 | 22 | нет | да | 15 | Отказать |
| Клиент 3 | 45 | да | нет | 80 | Одобрить |
| Клиент 4 | 19 | нет | нет | 10 | Отказать |
| Клиент 5 | 50 | да | да | 70 | Отказать |
| **Клиент 6** | 38 | да | нет | 55 | ??? |

**Разбор**

Клиент 6: работает=да, долги=нет → похож на Клиентов 1, 3. Модель предскажет: **Одобрить**.
Обрати внимание: «Клиент 5» – работает, доход 70, но есть долги → Отказать. Наличие долгов – «красный флаг» для модели.
Доход Клиента 6 также близок к доходу Клиента 1, но по этой маленькой таблице нельзя точно измерить вклад каждого признака.
Это учебный пример для разбора структуры данных. В реальных финансовых решениях требуется значительно больше проверенных данных, контроль ошибок и соблюдение установленных правил принятия решений.

## Задача В. Хорошая погода для прогулки?

| День | Темп. (°C) | Дождь? | Ветер (м/с) | Влажность (%) | МЕТКА |
|---|---|---|---|---|---|
| 1 | 18 | нет | 3 | 55 | Гулять |
| 2 | 8 | да | 12 | 90 | Не гулять |
| 3 | 22 | нет | 5 | 50 | Гулять |
| 4 | 15 | нет | 8 | 65 | Гулять |
| 5 | 12 | да | 15 | 85 | Не гулять |
| **6** | 20 | нет | 4 | 60 | ??? |

**Мини-проверка**

До чтения разбора сравни День 6 с остальными строками. На какие три дня он похож больше всего и по каким признакам?

---

**Разбор**

День 6: тепло (20°), нет дождя, слабый ветер (4), умеренная влажность (60%). Похоже на Дни 1, 3, 4. Модель предскажет: **Гулять**.
Главные признаки: «дождь» и «температура» – они сильнее всего разделяют классы. «Ветер» и «влажность» – дополнительные уточняющие признаки.
В этой таблице все четыре признака Дня 6 находятся ближе к примерам класса «Гулять», чем к примерам класса «Не гулять». Но вывод о главных признаках относится именно к представленным строкам. Если добавить, например, тёплые дождливые дни, граница между классами может стать сложнее.

# 6. От карточек – к таблицам данных

В реальных задачах машинного обучения обучающие примеры хранятся не в «карточках», а в таблицах – датасетах. Каждая строка – один объект. Каждый столбец – один признак. Последний столбец – метка (целевая переменная).
Такое представление позволяет отделить входные данные от ответа и передать их алгоритму в одинаковом порядке. Для каждого объекта число признаков должно соответствовать структуре, на которой обучалась модель.

## 6.1. Структура датасета

**Датасет – это «учебник» для модели**

```
Признак 1 Признак 2 Признак 3 ... Метка (Y)
__________ __________ __________ ________
18         нет         3          Гулять
8          да          12         Не гулять
22         нет         5          Гулять
...
```

- **X (матрица признаков)** – то, что модель «видит» на входе.
- **Y (вектор меток)** – правильные ответы, по которым модель учится.
- **Задача обучения:** найти функцию f(X) → Y.
- **Задача хорошего обучения:** найти f, которая верно работает и на НОВЫХ данных.

Если в X находится n строк и m признаков, размер матрицы X равен n × m. Вектор Y содержит n меток — по одной для каждой строки. Например, для 5 дней и 4 погодных признаков X имеет размер 5 × 4, а Y содержит 5 ответов.
Физически столбец с меткой может располагаться не только последним. Главное — правильно отделить его от признаков перед обучением.

## 6.2. Типы задач машинного обучения

Тип задачи определяется тем, какой результат требуется получить. Если ответ является категорией, нужна классификация; если числом — регрессия. В кластеризации заранее известных меток нет.

| Тип задачи | Метка Y | Примеры задач | Пример алгоритма |
|---|---|---|---|
| **Классификация** | Категория (кот / не кот, спам / не спам) | Диагностика, детекция, фильтрация | Дерево решений, нейросеть |
| **Регрессия** | Число (цена, температура, балл) | Прогноз цен, оценка рисков | Линейная регрессия |
| **Кластеризация** | Нет меток – алгоритм сам группирует | Сегментация покупателей | K-means |
| **Ранжирование** | Порядок (что показать первым) | Поиск, рекомендации | LambdaMART |

> **Обрати внимание**
>
> Число в данных не всегда означает регрессию. Если классы «спам» и «не спам» закодированы числами 1 и 0, задача остаётся классификацией.
> Определять тип задачи нужно по смыслу целевого ответа, а не только по способу его записи.

## 6.3. Что такое признак – подробнее

Алгоритм работает не с объектом «вообще», а с его представлением в данных. Поэтому выбор и подготовка признаков определяют, какую информацию модель сможет использовать.

| Тип признака | Что это | Примеры | Особенность для модели |
|---|---|---|---|
| **Числовой** | Измеримое число | Температура, возраст, цена | Можно сравнивать, считать среднее |

| Тип признака | Что это | Примеры | Особенность для модели |
|---|---|---|---|
| **Категориальный** | Одна из фиксированных категорий | Цвет, пол, город | Нужно преобразовать в числа (one-hot encoding) |
| **Бинарный** | Только 0 или 1 (да/нет) | Курит/нет, долги/нет | Частный случай категориального |
| **Текстовый** | Произвольный текст | Отзыв, письмо, tweet | Нужна дополнительная обработка (NLP) |
| **Временной** | Момент или период времени | Дата, час дня, день недели | Можно извлекать циклические признаки |

> **Частая ошибка**
>
> Нельзя считать любой числовой код количественной величиной. Например, запись «Москва = 1, Казань = 2, Омск = 3» не означает, что Омск «больше» Москвы.
> Именно поэтому категориальные признаки обычно кодируют специальным способом, например one-hot encoding: каждая категория получает отдельный бинарный признак.
> Также признак должен быть доступен в момент прогноза. Если в X случайно включить правильный ответ или информацию, появляющуюся только после события, проверка качества станет недостоверной.

## 6.4. Откуда берутся данные и почему их качество важно

Количество строк само по себе не гарантирует хорошее обучение. Модель извлекает закономерности именно из предоставленных данных, поэтому ошибки и перекосы выборки переходят в её прогнозы.

**Три проблемы реальных данных**

1. **ШУМНЫЕ ДАННЫЕ (Noise):** Ошибки при сборе – неверные метки, опечатки, сломанные датчики.
   *Проблема:* модель учится на ошибках и принимает их за закономерность.
   *Пример:* одинаковые объекты могут случайно получить разные метки. Чем больше таких противоречий, тем труднее модели найти устойчивую связь.
2. **СМЕЩЁННЫЕ ДАННЫЕ (Bias):** Данные не отражают реальное разнообразие – например, все обучающие фото котов сделаны при хорошем освещении. Модель не распознаёт котов в темноте.
   *Важно:* условия в обучающей выборке должны быть похожи на условия будущего применения модели.

3. **МАЛЫЙ ОБЪЁМ:** Слишком мало примеров → модель не видит разнообразия → ошибается на новых данных.
   *Правило:* чем сложнее задача – тем больше нужно примеров. Однако простое дублирование одних и тех же строк почти не добавляет новой информации. Полезны не только многочисленные, но и разнообразные примеры.

Перед обучением данные проверяют: ищут пропуски, невозможные значения, дубликаты и ошибки разметки.

# 7. Как это выглядит в Python

Ниже – минимальный код, который показывает полный цикл машинного обучения: от таблицы данных до прогноза. Цель не изучить sklearn, а увидеть, как понятия урока выглядят в коде. Строки матрицы X расположены в том же порядке, что и метки в y: первой строке X соответствует первая метка y, второй строке — вторая и так далее. Нарушение этого соответствия приведёт к неправильному обучению.

```
# Шаг 1. Данные (обучающие примеры)
# X – матрица признаков, y – метки
X = [
    [18, 0, 3, 55],  # Пример 1: темп, дождь(0/1), ветер, влажность
    [ 8, 1, 12, 90], # Пример 2
    [22, 0, 5, 50],  # Пример 3
    [15, 0, 8, 65],  # Пример 4
    [12, 1, 15, 85], # Пример 5
]
y = [1, 0, 1, 1, 0]  # 1 = гулять, 0 = не гулять

# Шаг 2. Обучение модели
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
model.fit(X, y)  # ← вот здесь происходит «обучение на примерах»

# Шаг 3. Прогноз для нового примера
new_day = [[20, 0, 4, 60]]  # температура=20, дождь=нет, ветер=4, влажность=60
prediction = model.predict(new_day)
print('Гулять?', 'Да' if prediction[0] == 1 else 'Нет')
# Вывод: Гулять? Да
```

*// X – матрица признаков.*
*// y – вектор меток.*
*// model.fit(X, y) – это и есть «обучение на примерах».*
*// model.predict(...) – прогноз для нового объекта без метки.*
В new_day признаки записаны в том же порядке, что и в каждой строке X: температура, наличие дождя, скорость ветра, влажность. Если поменять столбцы местами, модель будет неверно понимать входные числа.
Двойные квадратные скобки в new_day показывают, что методу predict передаётся набор объектов, состоящий здесь из одной строки. Результат prediction также является набором предсказаний, поэтому берётся его первый элемент prediction[0].
**Три строки, которые отражают всю суть урока**

- `model.fit(X, y)` → модель изучает закономерности из примеров
- `model.predict(new_day)` → модель применяет выученное к новым данным

Между fit() и predict() – вся «магия» машинного обучения. Что именно происходит внутри fit() – тема следующих уроков модуля.
Перед fit() данные должны быть подготовлены, а после обучения модель необходимо проверить на отдельных тестовых примерах. Без этой проверки единичный успешный прогноз ещё не доказывает качество модели.

**Мини-проверка**

Сколько обучающих объектов и сколько признаков у каждого объекта содержит X?
**Ответ:** 5 объектов и 4 признака. В y находится 5 соответствующих меток.
Какой информации нет у new_day? У него нет метки: именно её должна предсказать обученная модель.

# 8. Памятка

**Главные мысли урока**

1. Модель ИИ – не программа с готовыми правилами. Она ВЫВОДИТ правила из примеров.
2. Обучающий пример = набор признаков (X) + правильный ответ (Y = метка).
3. Признак – это одна измеримая характеристика объекта.
4. Обучение = поиск закономерностей в обучающей выборке.
5. Тестовая выборка – проверка на новых данных. Только она показывает реальное качество.
6. Качество модели зависит от качества данных: много + разнообразные + без ошибок.
7. Переобучение = «зазубрил учебник, провалил экзамен».

8. У нового объекта есть признаки, но нет известной модели метки: её заменяет прогноз.
9. Высокая уверенность модели не равна гарантии правильного ответа.
10. Больше данных полезно тогда, когда они добавляют разнообразие и правильно размечены.

**Проверь понимание**

1. В таблице «погода для прогулки» назови объект, четыре признака и метку.
2. Объясни, почему нельзя оценивать модель только по примерам, на которых она обучалась.
3. Определи тип задачи: предсказать точную температуру на завтра. Это регрессия, потому что целевой ответ — число.
4. Определи тип задачи: выбрать «дождь» или «без дождя». Это классификация, потому что ответ является категорией.

# Материалы к занятию

- AI_Learning_Blueprint.pptx (16.4 МБ)